Day 12 — Responsible AI, Experimentation, and Decision-Centered Evaluation

Beginner-friendly summary

A model is not good merely because its offline AUC, F1, or accuracy is high. A production model is useful only if it makes **better decisions under real constraints without causing unacceptable harm**.

For an expense-risk model, the full evaluation question is therefore:

> Does the model identify sufficiently valuable risky expenses, within investigator capacity, with acceptable false-positive cost, stable subgroup performance, appropriate governance, and evidence that deploying it actually improves business outcomes?

That requires several layers:

Layer	Main question
Offline evaluation	Is the model statistically better on historical unseen data?
Practical significance	Is the improvement large enough to matter?
Business evaluation	Does the threshold create positive expected value?
Responsible AI	Are errors distributed acceptably across relevant groups?
Shadow evaluation	Does the model behave safely on live traffic without affecting decisions?
Online experiment	Does using the model cause better outcomes?
Governance	Has the right evidence been reviewed and approved?
Monitoring	Does the model remain safe after launch?

A senior-level principle to remember is:

> **Prediction evaluation asks whether predictions are accurate. Experimentation asks whether using those predictions causes better outcomes. Responsible AI asks whether the resulting decision system is acceptable across affected populations and use cases.**

---

1. End-to-end mental model

```
Historical data
     |
     v
Point-in-time offline test
     |
     +--> Predictive metrics + confidence intervals
     +--> Paired model comparison
     +--> Business cost/value
     +--> Subgroup/fairness analysis
     |
     v
```

```
Shadow mode on live traffic
     |
     +--> Drift / latency / capacity / policy simulation
     |
     v
Controlled online experiment
     |
     +--> Primary business outcome
     +--> Guardrails
     +--> Fairness slices
     +--> Causal effect
     |
     v
Canary -> phased rollout -> full deployment
     |
     v
Monitoring + incidents + rollback + periodic review
```

The important separation is:

```
Offline:
"Would the model have predicted correctly?"

Online:
"Did deploying the model improve the decision process?"

Causal:
"Was the improvement actually caused by the intervention?"

Responsible AI:
"Was the resulting behavior acceptable across affected groups?"
```

---

2. Offline evaluation beyond one score

Suppose we have two expense-risk models:

```
Model A = current champion
Model B = proposed challenger
```

It is weak to say:

```
PR-AUC(A) = ...
PR-AUC(B) = ...

B is larger -> deploy B
```

You need to ask at least four questions:

1. How uncertain are those estimates?

2. Was the comparison made on exactly the same examples?
3. Is the improvement statistically credible?
4. Is the improvement practically valuable?

2.1 Point estimate versus confidence interval

A metric calculated from a finite test set is an estimate of unknown future performance.

Suppose:

$$\hat{M} = 0.42$$

The important question is not merely `0.42`.

You want something conceptually like:

$$95\%, CI = [L, U]$$

The interval communicates uncertainty.

A narrow interval:

```
------------------------
       [----]
```

suggests relatively precise estimation.

A wide interval:

```
------------------------
   [--------------]
```

suggests greater uncertainty.

For rare expense fraud or abuse, uncertainty can become large because there may be very few positive examples.

---

3. Bootstrap intuition

Bootstrap asks:

> What would our metric look like if we repeatedly obtained similar datasets from the underlying population?

Because we cannot repeatedly collect the past, we approximate this by sampling the test set **with replacement**.

Suppose the test set is:

```
A B C D E
```

One bootstrap sample could be:

```
B B E A C
```

Another:

```
D A D C E
```

For each resample:

```
calculate metric
```

After many repetitions:

```
metric_1
metric_2
metric_3
...
metric_B
```

The distribution approximates sampling uncertainty.

You can obtain percentile intervals, for example:

```
2.5th percentile ---- 97.5th percentile
```

for an approximate 95% bootstrap confidence interval.

Important correctness condition: bootstrap the right unit

Imagine one employee submits 50 expense claims.

Those 50 observations are correlated.

Naively resampling individual claims may pretend you have more independent observations than you really do.

Instead, potentially bootstrap:

```
employee
vendor
account
department
```

depending on the actual independence structure.

This is called **cluster bootstrap**.

Senior-level rule:

> Your statistical resampling unit should approximately preserve the dependency structure that generated the data.

---

4. Paired comparison of two models

When comparing two models, evaluate both on **the same observations**.

Wrong conceptual comparison:

```
Model A -> test sample X
Model B -> different test sample Y
```

Differences could come from the samples rather than models.

Better:

```
same test examples
       |
   +---+---+
   |       |
 Model A Model B
   |       |
 metricA metricB
       |
       v
metricB - metricA
```

For each bootstrap sample calculate:

$$\Delta_b = M_B^{(b)} - M_A^{(b)}$$

Then examine the distribution of:

$$\Delta$$

This is a **paired bootstrap**.

If almost the entire distribution is greater than zero, there is evidence B outperforms A on that metric.

But that still does not mean deployment is worthwhile.

---

5. Statistical versus practical significance

Imagine an enormous dataset produces:

$$\Delta AUC = 0.0002$$

with very small uncertainty.

It could be statistically significant while economically meaningless.

For production decisions define a **minimum practically meaningful improvement**.

For example conceptually:

```
Model B must achieve:

expected-value improvement >= business threshold
AND
review-capacity constraints satisfied
AND
critical guardrails not degraded
```

Do not invent the threshold after seeing experiment results.

Define it before evaluation where possible.

---

6. Permutation test intuition

Another approach to model comparison is a permutation/randomization test.

Under the null hypothesis:

> Model A and Model B are not meaningfully different.

For paired predictions, conceptually swap the A/B prediction assignment for examples and repeatedly calculate the resulting difference.

If the real difference is extremely unusual relative to these random differences, the null hypothesis becomes less plausible.

The intuition is:

```
Observed difference
       |
       v
Could this difference easily arise
if A and B were effectively interchangeable?
```

Bootstrapping is often used to estimate uncertainty.

Permutation testing is often used to test a null hypothesis.

They answer related but distinct questions.

---

7. Offline evaluation design for temporal finance data

For an expense-risk system, random splitting can create unrealistic results.

Prefer something resembling:

```
Training             Validation          Final Test
---------             ----------          ----------
past -----------------> later -------------> future
```

Example conceptually:

```
Train:      earlier periods
Validate:   subsequent period
Test:       newest untouched period
```

This better approximates deployment.

Also ensure:

```
features at prediction time
<=
information actually available at prediction time
```

Do not accidentally include:

- completed investigation outcome,
- future vendor statistics,
- future reimbursement status,
- downstream investigator decisions,
- post-submission edits unavailable when scoring occurred.

---

8. Online experimentation

Offline evaluation answers predictive questions.

A randomized controlled experiment can answer:

> Does introducing model-assisted review cause business outcomes to improve?

Suppose:

```
Control:
existing expense-review process

Treatment:
expense-review process + model ranking
```

Then compare downstream outcomes.

---

9. Randomization unit

This is one of the most important experiment-design decisions.

Potential randomization units include:

- individual expense claim,
- employee,
- reviewer,
- department,
- vendor,

- organization.

Consider contamination.

If claims from the same employee are split:

```
Claim 1 -> treatment
Claim 2 -> control
```

an investigator may learn information from Claim 1 that influences Claim 2.

Then the groups are no longer independent.

You might instead randomize at:

```
employee level
```

or potentially reviewer/team level.

But clustering decreases effective sample size.

So there is a trade-off:

```
smaller unit
    -> more statistical power
    -> greater contamination risk

larger unit
    -> less contamination
    -> lower effective sample size
```

Choose based on the actual mechanism.

---

10. Sample-ratio mismatch

Suppose the experiment intends:

```
50% control
50% treatment
```

but observes:

```
44% control
56% treatment
```

This may indicate **sample-ratio mismatch**, or SRM.

Potential causes include:

- faulty assignment logic,
- filtering after assignment,

- logging failures,
- treatment-specific eligibility,
- geographic routing,
- request failures,
- bots/retries,
- missing observations.

An experiment with unexplained SRM should not simply be analyzed as normal.

SRM is often a **data-quality alarm for experimentation**.

---

11. Statistical power

Power is the probability of detecting an effect of meaningful size when it truly exists.

It depends on:

$$\begin{aligned}\text{power} = f(&\\ &\text{sample size},\\ &\text{effect size},\\ &\text{variance},\\ &\alpha\\ &)\end{aligned}$$

Rare-event outcomes create problems.

For example:

```
confirmed severe expense abuse
```

may occur too rarely to produce adequate power quickly.

You might therefore have:

```
Primary metric:
high-value confirmed findings per reviewed case

Long-term outcome:
confirmed monetary loss avoided
```

But proxy metrics require validation because optimizing a proxy can create unintended behavior.

---

12. Primary metrics and guardrails

A production experiment should not optimize only one outcome.

Suppose treatment increases detected suspicious expenses but causes:

```
5x investigator workload
```

That might be unacceptable.

Define:

Primary outcome

The metric representing intended benefit.

Conceptually:

```
confirmed valuable risky cases detected
```

Guardrails

Metrics that must not degrade beyond predefined limits.

Potential categories:

```
false-positive burden
review queue size
review turnaround time
appeal/dispute rate
latency
system errors
subgroup harm
employee friction
investigator workload
```

The exact metrics depend on business policy.

---

13. Novelty effects

A treatment may perform differently when first introduced.

Investigators might initially:

- over-trust it,
- under-trust it,
- investigate unusually carefully,
- change behavior because they know the system is new.

Therefore:

```
Week 1 effect
```

may not equal:

```
steady-state effect
```

Analyze whether performance stabilizes.

---

14. Network effects and interference

Classical A/B testing commonly assumes one person's treatment does not affect another person's outcome.

This assumption is related to **SUTVA**.

But consider vendor risk.

If the model identifies one suspicious vendor and investigators block that vendor, then control employees interacting with the vendor are also affected.

Treatment spills over into control.

This is **interference**.

Possible approaches include:

```
cluster randomization
geographic randomization
department randomization
vendor-level randomization
switchback experiments
```

depending on the system.

---

15. Delayed outcomes

Many financial risk labels are delayed.

Example:

```
expense submitted
      |
      +---- model prediction
      |
      +---- investigation
      |
      +---- resolution
      |
      +---- recovery
```

The final resolution might arrive weeks later.

Therefore do not prematurely optimize on:

```
fast but weak proxy
```

when the true outcome is:

```
delayed but decision-relevant result
```

Maintain maturity windows.

For example conceptually:

```
Only evaluate confirmed-loss metric
for claims old enough to have completed
the normal investigation window.
```

---

16. Sequential testing

A dangerous pattern is:

```
Day 1: p = .20 -> continue
Day 2: p = .10 -> continue
Day 3: p = .04 -> STOP, winner!
```

Repeatedly checking ordinary fixed-horizon p-values inflates false-positive risk.

If you need continuous monitoring, use a method designed for sequential decisions, such as:

- group-sequential testing,
- alpha spending,
- always-valid confidence sequences/e-values where appropriate.

The principle is more important than memorizing one method:

> The stopping rule must be part of the statistical design.

Operational safety monitoring is different.

You should still monitor severe guardrails continuously and stop for safety incidents even if efficacy statistics are not mature.

---

17. Multiple comparisons

Imagine evaluating:

```
20 metrics
20 regions
10 departments
10 employee segments
```

If you search long enough, something will appear statistically unusual by chance.

Control this by separating:

```
Pre-specified confirmatory analysis
```

from:

```
Exploratory analysis
```

For confirmatory families, possible techniques include Bonferroni or Holm.

For exploratory discovery across many hypotheses, false discovery rate methods such as Benjamini-Hochberg may be appropriate.

The most important control is often:

> Pre-register the primary metric, important secondary metrics, subgroup hypotheses, and decision rule.

---

18. CUPED and variance reduction

Suppose employees historically have very different expense patterns.

A pre-experiment measurement can explain some future variance.

CUPED uses a covariate measured **before treatment**.

Conceptually:

$$Y_{adjusted} = Y - \theta(X - \bar{X})$$

where:

- (Y) = experiment outcome,
- (X) = correlated pre-treatment variable,
- (θ) = adjustment coefficient.

If (X) strongly predicts (Y), variance can decrease.

That means:

```
same underlying treatment effect
+
smaller noise
=
better statistical precision
```

Critical condition:

> The CUPED covariate must not itself be affected by treatment.

---

19. Shadow mode

Before allowing a risk model to alter decisions:

```
live traffic
     |
```

```
     +------> current production decision
     |
     +------> new model prediction
                    |
                    X no action
```

This is **shadow deployment**.

It allows you to evaluate:

- score distributions,
- latency,
- failure rates,
- drift,
- review-capacity implications,
- subgroup differences,
- disagreement with existing rules,
- operational robustness.

It does **not** establish treatment effect because the shadow model does not affect decisions.

---

20. Champion/challenger

```
Champion
= currently approved production system

Challenger
= candidate replacement
```

Compare both under identical traffic when possible.

A challenger should not win solely because of one metric.

Consider:

```
prediction quality
business value
fairness
latency
stability
cost
maintainability
explainability
governance implications
```

---

21. Canary rollout

After shadow validation:

```
New model
   |
 small production fraction
   |
 monitor
```

Example conceptually:

```
small cohort -> larger cohort -> broader cohort -> full
```

Actual rollout percentages should be chosen by the organization rather than assumed universally.

The purpose is blast-radius reduction.

---

22. Phased rollout and rollback thresholds

Before rollout, define explicit rollback criteria.

For example conceptually:

```
Rollback if:

critical latency > approved threshold
OR
system-error rate > threshold
OR
false-positive burden > threshold
OR
review backlog exceeds capacity
OR
critical subgroup guardrail fails
OR
serious policy/compliance incident occurs
```

Do not wait until an incident occurs to decide what counts as unacceptable.

---

23. Causal inference for observational data

Sometimes randomization is impossible.

You might ask:

> Did the model-assisted process reduce expense losses after deployment?

Comparing:

```
before deployment
vs
after deployment
```

does not automatically establish causality.

Other things may have changed:

```
policy
seasonality
employee mix
economic environment
audit intensity
review staffing
expense volume
vendor population
```

These are potential confounders.

---

24. Confounding

Suppose:

```
high-risk departments
     |
     +--> receive stricter review
     |
     +--> have higher observed violation rate
```

If you compare strictly reviewed versus normally reviewed claims, review intensity is not randomly assigned.

The groups may differ before treatment.

That makes causal interpretation difficult.

---

25. Selection bias

Selection bias occurs when observed samples differ systematically from the target population.

For example:

```
Only investigated claims receive reliable fraud labels.
```

Then your labels represent:

```
investigated population
```

rather than:

```
all expenses
```

A model trained on these labels can learn the historical investigation policy.

This creates a dangerous loop:

```
old policy
   ↓
which cases receive labels
   ↓
training data
   ↓
new model
   ↓
future investigation policy
```

---

26. Propensity scores

The propensity score is:

$$e(X) = P(T = 1|X)$$

It estimates treatment probability given observed covariates.

It can support:

- matching,
- stratification,
- weighting,
- covariate adjustment.

But an important limitation is:

> Propensity methods adjust only for measured confounders.

They cannot automatically fix:

```
unknown confounders
unmeasured intent
incorrect variables
bad overlap
post-treatment variables
```

---

27. Positivity / overlap

Suppose executives always receive manual review while everyone else never does.

Then there are no comparable observations.

Conceptually:

```
P(Treatment | executive) = 1
```

There is no counterfactual evidence for untreated executives.

This violates overlap/positivity.

No statistical technique can fully manufacture missing counterfactual support.

---

28. Difference-in-differences

Difference-in-differences compares changes over time.

Suppose:

```
Treatment group:
before -> after

Control group:
before -> after
```

Estimate:

$$\frac{(\text{Treatment} * after - \text{Treatment} * before)}{(\text{Control} * after - \text{Control} * before)}$$

The key assumption is approximately:

> Without treatment, treatment and control groups would have followed parallel trends.

You should examine pre-treatment trends rather than simply assert this assumption.

---

29. Sensitivity limits

Causal conclusions should be expressed proportionally to assumptions.

Avoid:

> "The model reduced fraud by X."

when evidence is observational and confounding cannot be excluded.

Prefer:

> "Under the stated no-unmeasured-confounding/parallel-trends assumptions, the analysis estimates an association consistent with a treatment effect of ..."

Randomization usually provides substantially stronger causal identification.

---

30. Connecting predictions to business decisions

Suppose the model outputs:

$$p = P(\text{high-risk expense}|X)$$

A probability itself does nothing.

A policy transforms it into an action:

```
p < T1
  -> allow normal processing

T1 <= p < T2
  -> additional automated checks

p >= T2
  -> human review
```

The real product is therefore:

$$\text{Model} + \text{Thresholds} + \text{Workflow}$$

not merely the predictive model.

---

31. Expected cost/value

For a binary review decision, define conceptually:

- (V_{TP}): value of correctly identifying a risky expense,
- (C_{FP}): cost of unnecessary review,
- (C_{FN}): cost of missed risk,
- (C_R): investigator review cost.

Then threshold selection should consider expected value.

A simplified formulation:

$$\begin{aligned} EV = \\ TP \cdot V_{TP} \\ - - - - - - - - - - - - - - \\ \#\#FP \cdot C_{FP} \\ \#\#FN \cdot C_{FN} \\ N_{review} \cdot C_R \end{aligned}$$

Exact terms depend on the business.

This is usually more decision-relevant than maximizing F1 blindly.

---

32. Review capacity

Suppose investigators can review only:

[K]

expenses each day.

Then operationally the relevant question becomes:

> Among the top K cases ranked by the model, how much genuine risk do we capture?

Useful evaluation metrics include:

$$Precision@K$$

and:

$$Recall@K$$

where (K) corresponds to realistic review capacity.

This connects ML directly to operations.

---

33. Decision curves

Threshold performance can be viewed across a range of business preferences.

Conceptually compare:

```
review nobody
review everybody
review using model
```

at different thresholds.

Decision-curve analysis tries to represent whether the model provides net benefit compared with these default strategies.

Senior principle:

> Evaluate the **policy induced by the model**, not just the score ranking.

---

34. Fairness definitions

There is no single mathematical definition of fairness.

Different definitions encode different policy objectives.

Let:

```
A = protected/relevant group
Y = true outcome
Ŷ = predicted decision
```

34.1 Demographic parity

Requires approximately:

$$P(\hat{Y} = 1|A = a) = P(\hat{Y} = 1|A = b)$$

Meaning groups receive positive decisions at similar rates.

For expense-risk detection, if `positive` means "sent for investigation", this means similar investigation rates.

But if underlying risks legitimately differ, demographic parity can conflict with other objectives.

34.2 Equal opportunity

Requires similar true-positive rates:

$$P(\hat{Y} = 1|Y = 1, A = a) = P(\hat{Y} = 1|Y = 1, A = b)$$

Interpretation:

> Among truly risky cases, groups are detected at similar rates.

This focuses on false-negative differences.

34.3 Equalized odds

Requires similar:

$$TPR$$

and:

$$FPR$$

across groups.

So both:

```
ability to detect true risky cases
```

and:

```
likelihood of incorrectly flagging legitimate cases
```

should be comparable.

34.4 Calibration

Calibration asks:

> When the model assigns risk 0.7, does the outcome occur approximately 70% of the time?

Group calibration asks whether this relationship is valid across groups.

Conceptually:

$$P(Y = 1|\hat{p} = 0.7, A = a) \approx 0.7$$

for relevant groups.

---

35. Fairness metrics can conflict

If base rates differ between populations and predictions are imperfect, it may be mathematically impossible to simultaneously satisfy some combinations of:

```
calibration
equal FPR
equal FNR
equal selection rates
```

Therefore:

> Choosing a fairness criterion is partially a policy and governance decision, not merely a machine-learning optimization problem.

Technical teams should expose trade-offs clearly rather than silently choosing the moral or legal objective.

---

36. Disparate-impact awareness

A system may use seemingly neutral rules yet produce meaningfully different outcomes across groups.

You should therefore examine:

```
selection/review rates
false positives
false negatives
appeals
downstream consequences
```

by relevant populations.

Some organizations use statistical ratios such as an "80% rule" as one screening heuristic in certain contexts, but it should not be treated as a universal legal or fairness definition.

Legal interpretation depends on jurisdiction and use case.

---

37. Where bias enters

Bias can appear throughout the system.

Stage	Example
Sampling	Some populations are underrepresented
Labels	Historical reviewers investigate groups differently
Features	Proxy variables encode sensitive attributes
Measurement	Expense behavior measured differently by region
Model	Errors concentrate in rare populations

Stage	Example
Threshold	One global threshold creates uneven impact
Policy	Some departments face more intense investigation
Human review	Reviewers interpret alerts differently
Deployment	Feedback loops reinforce previous targeting

A critical insight is:

> Bias is a property of the entire decision system, not merely the ML algorithm.

---

38. Proxy variables

Removing a protected attribute does not guarantee fairness.

For example:

```
postal code
department
job title
location
travel pattern
```

may partially encode attributes you intended not to use.

Protected attributes can sometimes still be required in a restricted evaluation dataset to **measure fairness**, even when they are not appropriate model inputs.

Governance and privacy controls matter here.

---

39. Subgroup performance

Overall performance can hide severe failures.

Suppose:

```
Overall recall = acceptable
```

but:

```
Region X recall = poor
Rare expense category recall = poor
New employees = poor
```

Evaluate relevant slices such as:

```
region
department
expense type
```

```
expense size band
employee tenure
vendor category
submission channel
policy version
risk band
```

Sensitive demographic slices should be included when legally and organizationally appropriate.

---

40. Intersectional slices

Bias can be hidden even within subgroup results.

For example:

```
Group A -> acceptable
Region X -> acceptable
```

does not imply:

```
Group A AND Region X -> acceptable
```

Intersectional groups may reveal failures.

But this creates a statistical problem.

---

41. Minimum support and uncertainty

Suppose:

```
Group A:
n = 100,000

Group B:
n = 22
```

A metric for Group B may be extremely noisy.

Do not report:

```
Recall = 0.67
```

without support information.

Report:

```
n
positive count
```

```
metric
confidence interval
```

Potential approaches include:

- minimum-support rules,
- confidence intervals,
- hierarchical/multilevel estimation where appropriate,
- aggregating compatible groups when justified,
- qualitative review.

Never interpret "no statistically significant disparity" as proof of fairness when the subgroup sample is simply too small.

---

42. Cost-sensitive decisioning

The costs of mistakes often differ.

Expense-risk example:

```
False positive
-> unnecessary investigation
-> employee friction
-> investigator cost

False negative
-> risky expense missed
-> potential financial loss
```

If:

$$C_{FN} \gg C_{FP}$$

you may choose a lower threshold.

But if review capacity is extremely constrained, lowering the threshold can flood the queue.

So:

$$\begin{aligned}\text{Threshold} = f(&\\ &\text{error cost},\\ &\text{capacity},\\ &\text{risk appetite},\\ &\text{fairness},\\ &\text{uncertainty}\\ &)\end{aligned}$$

---

43. Abstention

A model does not always have to decide.

You might define:

```
low risk
    -> normal process

high-confidence high risk
    -> review

uncertain
    -> abstain / alternate process
```

Abstention is particularly useful when:

- inputs are out of distribution,
- required data are missing,
- uncertainty is high,
- model disagreement is high,
- high-impact decisions require human judgment.

A senior system should have a strategy for:

> "I don't know."

---

44. Human-review thresholds

Human review is not free and is not automatically unbiased.

Evaluate:

```
cases/day generated
average review time
reviewer capacity
backlog
agreement between reviewers
overturn rate
model-human disagreement
```

A model that sends 10,000 alerts to a 500-case/day review team is operationally broken regardless of AUC.

---

45. Governance artifacts

Responsible deployment should leave an auditable evidence trail.

A mature system commonly needs artifacts corresponding to:

Artifact	Purpose
Model inventory entry	Know that the system exists and who owns it
Data card	Document data origin, coverage, limitations

Artifact	Purpose
Model card	Intended use, metrics, limitations, prohibited use
Risk tier	Determine required governance rigor
Validation report	Independent/second-line evidence where required
Approval matrix	Define who can approve deployment
Change record	Document what changed and why
Experiment record	Preserve hypotheses and test results
Monitoring plan	Define ongoing controls
Incident procedure	Define escalation and rollback

---

46. Risk tier

Not every model requires identical controls.

Risk can depend on:

```
decision impact
financial exposure
number of affected people
automation level
regulatory implications
reversibility
model complexity
use of sensitive data
```

For example:

```
Low-impact ranking recommendation
```

might receive lighter controls than:

```
fully automated employee disciplinary decision
```

The latter would generally require far stronger scrutiny.

---

47. Validation sign-off

The developer of the model should not be the only person assessing whether it is appropriate to launch.

Possible governance roles conceptually include:

```
Model owner
Engineering owner
Business/process owner
```

```
Risk/model validation
Compliance/legal
Security/privacy
Operations
Responsible-AI review
```

The exact approval matrix depends on the organization.

---

48. Change control

A production model is not only:

```
model.pkl
```

A meaningful change may include:

```
new model
new feature
new dataset
new threshold
new label definition
new policy
new LLM prompt
new review workflow
new geography
new user population
```

Any of these may invalidate previous validation.

Therefore version:

```
data
features
model
calibration
threshold
policy
code
evaluation
approvals
```

---

49. Monitoring

Production monitoring should cover more than predictive accuracy.

Think in layers:

System

```
latency
errors
throughput
availability
```

Data

```
schema
missingness
category changes
volume
distribution drift
```

Model

```
score distribution
calibration
precision
recall
PR-AUC
```

when labels mature.

Decision

```
review volume
accept/reject rate
overturn rate
queue backlog
```

Responsible AI

```
subgroup performance
selection rate
FPR/FNR differences
```

Business

```
confirmed findings
avoidable loss
review cost
employee friction
```

---

50. Incident triggers

Examples of incident conditions should be decided in advance.

Conceptually:

```
critical schema corruption
major model-performance degradation
unexpected concentration of alerts
subgroup guardrail breach
review queue overload
sudden calibration failure
security/privacy event
severe incorrect automated action
```

Each should map to:

```
severity
owner
notification
mitigation
rollback
postmortem
```

---

51. Rollback versus deactivation

Rollback:

```
Model version B
      ↓
restore previous approved Model A
```

Deactivation:

```
model-assisted decisioning
      ↓
disable
      ↓
fallback rules/manual process
```

A production architecture needs a fallback before launch.

Otherwise a rollback plan exists only on paper.

---

52. Post-deployment review

Passing the launch gate is not permanent approval.

Review after:

```
sufficient mature labels
major distribution shift
policy change
```

```
new geography
new population
new feature
serious incident
periodic governance cycle
```

You may discover that the model remains accurate but the business process around it has changed.

Practical task — Expense-risk model evaluation plan

Assume an expense-risk model produces:

```
risk_score = P(expense requires investigation)
```

and human investigators have limited daily capacity.

No real company metrics are assumed below; placeholders such as `<REVIEW_CAPACITY>` should be replaced by actual approved values.

---

53. Shareable thought process / decision framework

Before writing code, I would structure the problem this way.

Step 1 — Define the actual decision

Not:

```
Predict risky expense.
```

Instead:

```
Which expenses should be sent to human review,
given limited investigator capacity?
```

This immediately determines important metrics.

Step 2 — Define the outcome

Specify:

```
What exactly counts as "risky"?
When is that outcome considered mature?
Who assigns the label?
Could the label depend on the previous review policy?
```

Without a defensible label, sophisticated experimentation does not rescue the system.

Step 3 — Establish an untouched temporal benchmark

Use historical data available at the actual scoring time.

Compare:

```
existing process
baseline model
candidate model
```

on the same evaluation population.

Step 4 — Quantify uncertainty

For important metrics report:

```
point estimate
confidence interval
paired difference
confidence interval of difference
```

not merely one number.

Step 5 — Convert ranking into decisions

Given review capacity:

$$K = \text{approved review capacity}$$

evaluate top-K behavior.

Step 6 — Examine subgroup behavior

For every governance-approved slice:

```
sample size
positive support
precision
recall
FPR
calibration
review rate
uncertainty
```

Step 7 — Shadow the model

Run on live traffic without altering decisions.

Validate:

```
latency
stability
traffic coverage
score distribution
drift
capacity
subgroup mix
```

Step 8 — Run causal experiment

Randomize an appropriate unit.

Measure:

```
business benefit
review burden
fairness guardrails
operational guardrails
```

Step 9 — Canary and phase rollout

Scale only if evidence remains acceptable.

Step 10 — Monitor continuously

Maintain rollback/deactivation mechanisms and mature-label evaluation.

---

54. Evaluation plan

Phase A — Offline

Dataset

```
Train:
historical period

Validation:
later historical period

Final evaluation:
most recent untouched mature-label period
```

Point-in-time correctness is mandatory.

Candidate comparison

Compare:

```
Current champion
vs
New challenger
```

on identical examples.

Core predictive metrics

Because expense risk may be rare, prioritize metrics such as:

```
PR-AUC
Precision@K
Recall@K
precision/recall at operating threshold
calibration
Brier score
```

ROC-AUC can remain useful but should not be the only metric.

Uncertainty

Use paired cluster bootstrap if multiple claims per employee/vendor create dependence.

Business evaluation

For each threshold calculate:

```
review count
true positives
false positives
false negatives
expected investigation cost
expected prevented/recovered value
```

using business-approved cost assumptions.

---

55. Capacity-aware threshold

Suppose:

```
review_capacity = <REVIEW_CAPACITY_PER_DAY>
```

For each day's expenses:

```
sort by risk descending
select top review_capacity
```

Then evaluate the consequences.

This avoids choosing a threshold in isolation from queue capacity.

A fixed threshold can still be used, but you need to understand what happens if daily score distributions change.

---

56. Fairness/subgroup matrix

A useful evaluation artifact could look like:

Slice	Support	Positive support	Review rate	Precision	Recall/TPR	FPR	Calibration	CI
Overall	<n>	<n+>	<...>	<...>	<...>	<...>	<...>	<...>
Region A	...	...	...	...	...	...	...	...
Region B	...	...	...	...	...	...	...	...
Expense type X	...	...	...	...	...	...	...	...
Group intersections	...	...	...	...	...	...	...	...

For small support:

```
do not overinterpret point estimate
```

and explicitly mark:

```
insufficient evidence / wide uncertainty
```

---

57. Fairness policy statement

Do not let engineering silently choose the fairness objective.

Document something like:

```
The system is evaluated for [approved fairness objectives]
because false-positive investigation and false-negative
missed-risk consequences have been assessed as [policy rationale].

No claim is made that satisfying these metrics establishes
fairness in every normative or legal sense.
```

The actual wording should be approved by relevant governance functions.

---

58. Causal assumptions

For the online experiment document:

Treatment

```
Model-assisted prioritization available to reviewers.
```

Control

```
Current approved prioritization process.
```

Randomization unit

Potentially employee or another contamination-resistant unit, after studying operational interference.

Assumptions

Document:

```
randomization implemented correctly
no material unmeasured differential attrition
limited interference between units
outcomes defined consistently
label maturation handled identically
treatment does not alter logging
```

If these fail, causal interpretation weakens.

---

59. Shadow evaluation

Before experiment treatment affects humans:

```
Incoming expense
       |
       +------> current system
       |
       +------> challenger score
                        |
                        v
                  shadow logs
```

Gate criteria should cover:

```
service reliability
latency
missing features
out-of-distribution traffic
score stability
review-volume simulation
subgroup behavior
```

No launch based solely on historical accuracy.

60. Online experiment

Example structure:

```
Eligible units
      |
random assignment
   /         \
Control    Treatment
   |           |
existing    model-assisted
process     process
   \          /
    downstream outcomes
           |
    causal comparison
```

Pre-register:

```
primary metric
guardrails
minimum meaningful effect
analysis window
maturity window
stopping rule
exclusions
subgroup analyses
```

61. Online metric hierarchy

Conceptually:

Primary

```
Business-value outcome per eligible expense
```

or another business-approved decision metric.

Secondary

```
confirmed high-risk detection
precision among reviewed cases
review efficiency
```

Operational guardrails

```
latency
errors
```

```
backlog
reviewer workload
```

Human-impact guardrails

```
false-positive investigation burden
appeals
overturns
```

Responsible-AI guardrails

```
approved subgroup disparities
```

Do not invent acceptable limits.

Store them as:

```
<MAX_ALLOWED_FPR_GAP>
<MAX_REVIEW_RATE_RATIO>
<MAX_QUEUE_BACKLOG>
```

until business/governance owners approve values.

---

62. Rollout gates

A deployment decision could follow:

```
Offline validation passed
        |
        v
Shadow criteria passed
        |
        v
Governance approval
        |
        v
Controlled experiment
        |
        +--> no-go -> stop / revise
        |
        v
Canary
        |
        +--> rollback on threshold breach
        |
        v
Phased rollout
        |
```

```
        v
Full rollout
```

---

63. Rollback criteria

Define before canary.

Conceptually:

```
Rollback immediately:
- severe privacy/security incident
- incorrect irreversible action
- critical system failure

Rollback or halt rollout:
- review load above approved capacity
- primary outcome materially worse
- false-positive burden beyond guardrail
- subgroup safety/fairness guardrail breached
- calibration/performance degradation beyond approved band
- unexplained SRM or experiment integrity failure
```

The numeric limits belong in the approved operational policy.

---

64. Governance sign-offs

A deployment record could require:

Gate	Typical accountable function
Problem/use-case approval	Business owner
Data suitability	Data owner/governance
Technical validation	ML engineering/data science
Independent validation	Model-risk/validation function where required
Fairness assessment	Responsible-AI/risk/compliance
Privacy	Privacy/legal
Security	Security
Operational readiness	SRE/platform/operations
Final production approval	Defined approval authority

The organization may combine or rename these roles.

---

65. Pseudocode before implementation

```
INPUT:
    evaluation dataset
```

```
    champion predictions
    challenger predictions
    labels
    entity IDs
    timestamps
    subgroup columns
    review capacity
    approved business costs

1. Validate data
    check label maturity
    check timestamps
    check missing values
    check point-in-time correctness

2. Evaluate champion and challenger
    PR-AUC
    ROC-AUC
    calibration
    Brier score

3. Perform paired bootstrap
    resample correct independent unit
    calculate challenger_metric - champion_metric
    estimate confidence interval

4. Apply operational decision policy
    for each day:
        rank expenses by risk score
        select at most review_capacity cases

5. Evaluate decision metrics
    precision@capacity
    recall@capacity
    false positives
    false negatives
    review volume
    expected cost/value

6. Evaluate slices
    for each subgroup:
        calculate support
        positive support
        recall
        FPR
        precision
        review rate
        uncertainty

7. Flag low-support slices
    do not treat unstable point estimates
```

```
    as reliable evidence

8. Produce offline validation artifact

9. Shadow deployment
    log scores without changing actions
    verify latency, drift, capacity, fairness

10. Online experiment
    randomize approved unit
    verify SRM
    measure primary metric and guardrails
    follow pre-specified sequential-testing rule

11. If all launch gates pass
    canary rollout

12. During rollout
    monitor rollback thresholds

13. After label maturation
    conduct post-deployment review
```

---

66. Python evaluation scaffold

This intentionally produces no invented business result. It provides components you can connect to a real or synthetic dataset.

```
from dataclasses import dataclass
from typing import Sequence

import numpy as np
import pandas as pd

from sklearn.metrics import (
    average_precision_score,
    roc_auc_score,
    brier_score_loss,
    confusion_matrix,
)


@dataclass(frozen=True)
class CapacityMetrics:
    reviewed: int
    precision: float
    recall: float
    true_positives: int
    false_positives: int
    false_negatives: int
```

```
def predictive_metrics(
    y_true: np.ndarray,
    y_score: np.ndarray,
) -> dict[str, float]:
    return {
        "pr_auc": average_precision_score(y_true, y_score),
        "roc_auc": roc_auc_score(y_true, y_score),
        "brier_score": brier_score_loss(y_true, y_score),
    }


def metrics_at_capacity(
    y_true: np.ndarray,
    y_score: np.ndarray,
    review_capacity: int,
) -> CapacityMetrics:
    if review_capacity < 0:
        raise ValueError("review_capacity must be non-negative")

    n = len(y_true)

    if len(y_score) != n:
        raise ValueError("y_true and y_score must have equal length")

    k = min(review_capacity, n)

    order = np.argsort(-y_score)
    selected = order[:k]

    reviewed_mask = np.zeros(n, dtype=bool)
    reviewed_mask[selected] = True

    tp = int(np.sum((y_true == 1) & reviewed_mask))
    fp = int(np.sum((y_true == 0) & reviewed_mask))
    fn = int(np.sum((y_true == 1) & ~reviewed_mask))

    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0

    return CapacityMetrics(
        reviewed=k,
        precision=precision,
        recall=recall,
        true_positives=tp,
        false_positives=fp,
        false_negatives=fn,
    )
```

```
def paired_bootstrap_difference(
    y_true: np.ndarray,
    champion_score: np.ndarray,
    challenger_score: np.ndarray,
    metric_fn,
    n_bootstrap: int = 2000,
    seed: int = 42,
) -> dict[str, float]:
    """
    Simple row-level paired bootstrap.

    For correlated observations such as multiple expenses per employee,
    replace this with an entity/cluster bootstrap.
    """
    rng = np.random.default_rng(seed)
    n = len(y_true)

    diffs = []

    for _ in range(n_bootstrap):
        idx = rng.integers(0, n, size=n)

        y_b = y_true[idx]
        champion_b = champion_score[idx]
        challenger_b = challenger_score[idx]

        # Skip pathological resamples with one class only.
        if len(np.unique(y_b)) < 2:
            continue

        champion_metric = metric_fn(y_b, champion_b)
        challenger_metric = metric_fn(y_b, challenger_b)

        diffs.append(challenger_metric - champion_metric)

    diffs = np.asarray(diffs)

    if len(diffs) == 0:
        raise ValueError("No valid bootstrap samples were produced")

    return {
        "mean_difference": float(np.mean(diffs)),
        "ci_2.5": float(np.percentile(diffs, 2.5)),
        "ci_97.5": float(np.percentile(diffs, 97.5)),
    }


def subgroup_binary_metrics(
    df: pd.DataFrame,
    group_column: str,
    label_column: str,
```

```
    score_column: str,
    threshold: float,
) -> pd.DataFrame:

    rows = []

    for group_value, group_df in df.groupby(group_column, dropna=False):
        y_true = group_df[label_column].to_numpy()
        y_score = group_df[score_column].to_numpy()

        y_pred = (y_score >= threshold).astype(int)

        tn, fp, fn, tp = confusion_matrix(
            y_true,
            y_pred,
            labels=[0, 1],
        ).ravel()

        positive_support = tp + fn
        negative_support = tn + fp

        tpr = tp / positive_support if positive_support else np.nan
        fpr = fp / negative_support if negative_support else np.nan
        precision = tp / (tp + fp) if (tp + fp) else np.nan
        review_rate = np.mean(y_pred)

        rows.append({
            "group": group_value,
            "support": len(group_df),
            "positive_support": int(positive_support),
            "precision": precision,
            "tpr_recall": tpr,
            "fpr": fpr,
            "review_rate": review_rate,
        })

    return pd.DataFrame(rows)


def daily_capacity_policy(
    df: pd.DataFrame,
    date_column: str,
    score_column: str,
    capacity_per_day: int,
) -> pd.Series:
    """
    Returns a Boolean Series indicating which expenses receive review.
    """

    if capacity_per_day < 0:
        raise ValueError("capacity_per_day must be non-negative")
```

```
    selected = pd.Series(False, index=df.index)

    for _, day_df in df.groupby(date_column):
        top_indices = (
            day_df
            .sort_values(score_column, ascending=False)
            .head(capacity_per_day)
            .index
        )

        selected.loc[top_indices] = True

    return selected
```

---

67. Non-obvious implementation logic

The important part is not the syntax.

Paired bootstrap

Both models use the same resampled indices:

```
champion_b = champion_score[idx]
challenger_b = challenger_score[idx]
```

This preserves pairing.

If you sampled them separately, you would inject unnecessary variance and weaken the comparison.

Capacity policy

We rank independently per day:

```
groupby(date)
```

because a review team often operates with a per-period capacity.

Selecting the top 1,000 examples across an entire three-month dataset would not faithfully simulate daily operations.

`min(capacity, n)`

Some days may contain fewer expenses than the nominal review capacity.

The system must handle that safely.

Low-support subgroups

The function returns support rather than hiding it.

A production evaluation should additionally calculate uncertainty rather than making confident conclusions from tiny groups.

Brier score

Ranking metrics do not tell you whether probabilities are trustworthy.

If downstream policies depend on:

```
risk_score = 0.8
```

calibration matters.

---

68. Production trade-offs

A. Global threshold versus top-K

Global threshold

```
review if p >= 0.72
```

Advantages:

- stable interpretation,
- easier policy documentation.

Problems:

- workload varies with score distribution.

Top-K

```
review highest K scores
```

Advantages:

- capacity controlled directly.

Problems:

- meaning of the lowest reviewed risk changes over time.

A hybrid strategy is often possible:

```
review if:
    score >= minimum-risk threshold
AND
    capacity allows
```

B. Fairness versus capacity

Suppose fixing a subgroup recall gap requires more reviews.

But reviewers are already at capacity.

You then have a real policy trade-off:

```
capacity
fairness objective
financial risk
employee impact
```

There is no model metric that automatically resolves the policy decision.

Escalate the trade-off transparently.

C. Precision versus recall

Higher threshold:

```
higher precision
lower review volume
more risky cases missed
```

Lower threshold:

```
higher recall
more false positives
greater review workload
```

The appropriate point depends on the relative consequences.

---

69. Important failure modes

1. Optimizing only AUC

The new model wins AUC but produces a worse top-of-queue ranking.

Result:

```
better benchmark
worse operations
```

---

2. Ignoring confidence intervals

A tiny subgroup appears to improve dramatically due to sampling noise.

---

3. Ignoring temporal drift

Historical vendor patterns disappear after policy changes.

4. Treating observational associations as causal

Deployment and lower fraud coincide, but a new audit policy began simultaneously.

---

5. Threshold tuned on final test data

The supposedly untouched test set becomes part of training/model selection.

---

6. Review capacity omitted

A high-recall model overwhelms investigators.

---

7. Historical labels treated as objective truth

Labels encode previous investigation policies.

---

8. Fairness evaluated only globally

A severe intersectional failure remains invisible.

---

9. Small groups overinterpreted

A subgroup with very few positive examples produces unstable recall estimates.

---

10. Shadow results treated as causal evidence

Shadow mode tests live technical behavior, not business treatment effect.

---

11. Experiment contamination

Reviewers use information from treatment cases when handling control cases.

---

12. Peeking at p-values

The team stops once significance is reached without sequential-testing correction.

---

13. Hundreds of subgroup tests

Some disparities appear significant purely by chance.

---

14. Missing rollback path

The model is technically reversible but the business workflow cannot revert.

---

70. Communicating fairness conclusions correctly

Bad:

> "The model is fair because demographic parity passed."

Better:

> "Under the organization's selected fairness criteria, evaluated on the available sample, we did not observe disparities beyond the approved limits for the tested groups. The conclusion is limited by subgroup support, label quality, the chosen fairness definition, and unobserved downstream effects."

Similarly for causality.

Bad:

> "Fraud decreased after deployment, proving the model worked."

Better:

> "The randomized experiment estimates the treatment effect of model-assisted review under the experiment's randomization, interference, outcome-definition, and compliance assumptions."

For observational analysis:

> "The estimated effect additionally depends on assumptions regarding measured confounding, overlap, and the chosen identification strategy."

This is not hedging. It is scientifically correct communication.

---

71. Senior-level synthesis

When asked how you would evaluate an expense-risk system, do not start with:

> "I would measure AUC."

Start with the decision architecture:

```
Business decision
      |
      v
What action follows the score?
      |
      v
What are costs and review capacity?
      |
      v
Offline ranking + calibration + uncertainty
      |
      v
Subgroup/fairness analysis
      |
      v
Shadow deployment
      |
      v
Randomized experiment
```

```
      |
      v
Business outcome + guardrails
      |
      v
Governance approval
      |
      v
Canary/phased rollout
      |
      v
Continuous monitoring and rollback
```

The strongest conceptual distinction for Day 12 is:

$$\boxed{\begin{aligned}&\text{Good model}\\ &\neq\\ &\text{Good decision system}\end{aligned}}$$

Instead:

$$\boxed{\begin{aligned}\text{Good decision system} = {}&\text{prediction quality}\\ &+ \text{causal impact}\\ &+ \text{business value}\\ &+ \text{capacity constraints}\\ &+ \text{fairness}\\ &+ \text{governance}\\ &+ \text{monitoring}\end{aligned}}$$

And two final principles are worth carrying into senior Applied AI/ML design discussions:

> **Fairness metrics encode policy choices. Engineering should quantify the consequences and uncertainty, but should not silently decide which definition society or the organization ought to optimize.**

> **Causal conclusions are only as strong as the design and assumptions supporting identification. Predictive correlation, temporal association, and causal effect are three different things.**

Day 12 DSA — BFS: Breadth-First Search

Beginner-friendly summary

BFS explores nodes level by level. It uses a **queue (FIFO)** so that nodes discovered earlier are processed earlier.

BFS is especially useful for:

- tree **level-order traversal**,
- graph traversal,

- **shortest path in an unweighted graph**,
- minimum-number-of-steps problems,
- grid problems where every move has the same cost,
- finding everything reachable within `K` moves.

Core pattern:

```
Start
  |
  v
Queue [source]
  |
  v
Process nearest nodes first
  |
  v
Then nodes 1 step farther
  |
  v
Then nodes 2 steps farther
```

The key interview fact is:

> **BFS gives shortest-path distance when every edge has equal cost.**

---

1. Recognition signals

Think **BFS** when the problem contains phrases such as:

- level by level,
- minimum number of moves,
- shortest path in an **unweighted** graph,
- nearest node/cell,
- fewest transformations,
- minimum hops,
- minimum edges,
- spread/infection over time,
- distance from a source,
- all nodes within `K` steps.

Examples:

```
Tree:
"Return values level by level."

Graph:
"Find the minimum number of flights if every flight counts as one hop."

Grid:
```

```
"Find the shortest route from top-left to bottom-right."

Transformation:
"Minimum number of word changes."
```

---

2. BFS versus DFS recognition

Suppose:

```
A -- B -- D
|
C -- E -- F
```

DFS might explore:

```
A -> B -> D
```

before exploring `C`.

BFS explores:

```
Level 0: A
Level 1: B, C
Level 2: D, E
Level 3: F
```

Therefore if you need:

```
minimum number of edges
```

BFS naturally processes candidates in increasing distance.

---

3. Core data structure: queue

Python normally uses:

```
from collections import deque
```

A BFS queue behaves as:

```
enqueue --> [ A B C ] --> dequeue
             FIFO
```

Use:

```
queue.append(value)
```

to enqueue and:

```
queue.popleft()
```

to dequeue.

Avoid:

```
list.pop(0)
```

because removing the first element of a Python list costs (O(n)).

`deque.popleft()` is (O(1)).

---

4. Queue invariants

This is the most important BFS reasoning concept.

An **invariant** is something that remains true while the algorithm runs.

For standard BFS:

Invariant 1 — FIFO ordering

Nodes are processed in the order they were discovered.

```
discover A
discover B
discover C

processing order:
A -> B -> C
```

Invariant 2 — Nondecreasing distance

If `u` is removed before `v`, BFS never processes a node with a larger known shortest distance before a smaller one.

Conceptually:

```
distance 0
    ↓
distance 1
    ↓
distance 2
    ↓
distance 3
```

Invariant 3 — First discovery is shortest

For an unweighted graph:

> The first time BFS reaches a node, it has found a shortest path to that node.

Therefore we normally mark a node visited **when adding it to the queue**, not when removing it.

Correct:

```
visited.add(neighbor)
queue.append(neighbor)
```

This prevents duplicate queue entries.

---

5. Why BFS finds shortest paths

Suppose:

```
S
| \
A  B
|  |
C  D
 \
  T
```

BFS starts at `S`.

```
Distance 0:
S

Distance 1:
A, B

Distance 2:
C, D

Distance 3:
T
```

Before processing any distance-3 node, BFS has already processed all reachable nodes at distances 0, 1, and 2.

Therefore the first time `T` is reached, there cannot be another path to `T` containing fewer edges that BFS somehow skipped.

That is the reason BFS provides shortest paths in unweighted graphs.

---

6. Important limitation

BFS is **not** generally the correct shortest-path algorithm when edges have different costs.

Example:

```
A --100--> B
|
1
v
C --1--> B
```

Counting edges:

```
A -> B
1 edge
```

But weighted cost:

```
A -> C -> B
1 + 1 = 2
```

The direct path costs `100`.

For non-negative weighted edges, think:

```
Dijkstra
```

For equal/unweighted edges, think:

```
BFS
```

---

7. BFS on a tree — level-order traversal

Consider:

```
        10
       /  \
      20   30
     / \    \
    40 50    60
```

BFS order:

```
10
20 30
40 50 60
```

The queue evolves as:

```
[10]

process 10
[20, 30]
```

```
process 20
[30, 40, 50]

process 30
[40, 50, 60]

...
```

Python tree BFS template

```
from collections import deque


def level_order(root):
    if root is None:
        return []

    queue = deque([root])
    result = []

    while queue:
        node = queue.popleft()

        result.append(node.val)

        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)

    return result
```

For a proper tree, a `visited` set is normally unnecessary because nodes do not point back to their parents unless the representation explicitly includes parent links.

---

8. Returning separate tree levels

Sometimes the required output is:

```
[
    [10],
    [20, 30],
    [40, 50, 60]
]
```

The important trick is:

```
level_size = len(queue)
```

At the beginning of an iteration, the queue contains exactly the nodes belonging to the current level.

```
from collections import deque


def level_order_by_level(root):
    if root is None:
        return []

    queue = deque([root])
    result = []

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()

            level.append(node.val)

            if node.left:
                queue.append(node.left)

            if node.right:
                queue.append(node.right)

        result.append(level)

    return result
```

---

9. Generic graph BFS

Consider an adjacency list:

```
graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["E"],
    "D": [],
    "E": []
}
```

Generic BFS:

```
from collections import deque


def bfs(graph, start):
    queue = deque([start])
    visited = {start}

    order = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order
```

Possible result:

```
A B C D E
```

---

10. Why `visited` matters for graphs

Suppose:

```
A ---- B
|      |
|      |
C ---- D
```

Without `visited`:

```
A -> B -> A -> C -> ...
```

Nodes can be rediscovered repeatedly.

With:

```
visited = {start}
```

each node enters the queue at most once.

---

11. Critical implementation choice

Compare these.

Risky pattern

```
node = queue.popleft()

if node in visited:
    continue

visited.add(node)
```

Several parents might add the same node before it is processed.

Example:

```
A -> C
B -> C
```

`C` could be placed in the queue twice.

Better BFS pattern

Mark visited when **enqueuing**:

```
if neighbor not in visited:
    visited.add(neighbor)
    queue.append(neighbor)
```

Now every node enters the queue at most once.

---

12. BFS shortest-path template

If we need the minimum number of edges:

```
from collections import deque


def shortest_distance(graph, start, target):
    queue = deque([(start, 0)])
    visited = {start}

    while queue:
        node, distance = queue.popleft()

        if node == target:
            return distance

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, distance + 1))
```

```
    return -1
```

The queue stores:

```
(node, distance)
```

For example:

```
(A, 0)
(B, 1)
(C, 1)
(D, 2)
```

---

13. Brute-force shortest-path reasoning

Before BFS, consider the brute-force idea.

Suppose we need:

```
shortest path from S to T
```

One approach is:

1. Generate every possible path.
2. Ignore paths that revisit nodes.
3. Calculate the length of each valid path reaching `T`.
4. Return the smallest.

Conceptually:

```
S
├── path 1
│   └── ...
├── path 2
│   └── ...
├── path 3
│   └── ...
└── ...
```

A graph can contain exponentially many simple paths.

So brute-force path enumeration can approach exponential complexity.

But notice what the question actually asks:

```
minimum number of edges
```

We don't need to enumerate long paths before checking shorter ones.

That observation leads directly to BFS.

---

14. Optimized reasoning

BFS explores:

```
all paths of length 0
then
all possibilities at distance 1
then
all possibilities at distance 2
...
```

So once the destination is first reached:

```
STOP
```

because any undiscovered path would be at least as long.

That converts potentially exponential path enumeration into:

[O(V+E)]

for adjacency-list graphs.

Medium Problem — Shortest Path in Binary Matrix

A useful BFS medium problem is:

> Given an `n x n` binary matrix, find the length of the shortest clear path from the top-left cell `(0,0)` to the bottom-right cell `(n-1,n-1)`.

A cell containing:

```
0 = open
1 = blocked
```

You may move in **8 directions**:

```
↖ ↑ ↗
← X →
↙ ↓ ↘
```

Return `-1` if no valid path exists.

---

15. Recognition signals

Immediately notice:

Signal 1

We need:

```
shortest path
```

Signal 2

Every movement costs exactly:

```
1 step
```

Signal 3

The grid can be interpreted as an unweighted graph.

Each open cell is a node.

```
cell
  |
adjacent open cells
```

Therefore:

> **Use BFS.**

---

16. Example

Consider:

```
0 1 0
0 0 0
1 0 0
```

Start:

```
S 1 0
0 0 0
1 0 T
```

A shortest path could be:

```
S
 \
  *
   \
    T
```

because diagonal moves are permitted.

---

17. Brute-force reasoning

A brute-force DFS could explore every possible path.

Pseudo-logic:

```
dfs(cell):

    if cell is destination:
        update shortest

    for every available neighbor:
        mark visited
        recurse
        unmark visited
```

The difficulty is that many different paths may reach the same cells.

The number of possible paths can grow exponentially.

Worst-case conceptual complexity is exponential.

DFS is therefore a poor natural choice for finding the shortest path in this unweighted grid.

---

18. Optimized BFS reasoning

Start from `(0,0)`.

The BFS levels represent path lengths:

```
Level 1:
start cell

Level 2:
cells reachable in one move

Level 3:
cells reachable in two moves

...
```

Therefore the first time we reach:

```
(n - 1, n - 1)
```

we have discovered a shortest path.

---

19. Edge cases

Before coding, identify these explicitly.

Empty/invalid matrix

Depending on the problem contract, handle appropriately.

Starting cell blocked

```
1 ...
```

No path exists.

Return:

```
-1
```

Ending cell blocked

```
...
... 1
```

No path exists.

Single-cell matrix

```
[0]
```

The start is already the destination.

The path length is:

```
1
```

Notice that the problem counts **cells in the path**, not edges.

No available route

Return:

```
-1
```

Cycles

Grid movement can return to a previously visited cell.

We must mark visited.

Diagonal moves

Do not accidentally use only four directions.

We need all eight.

---

20. Complexity

There are:

$$n^2$$

cells.

Each cell is processed at most once.

Each processing operation examines at most eight neighbors.

Therefore:

$$O(n^2)$$

time.

Visited/queue storage can contain up to (n^2) cells:

$$O(n^2)$$

space.

More generally, thinking of it as a graph:

[O(V+E)]

Since each grid cell has at most eight edges:

$$E = O(V)$$

so:

$$O(V + E) = O(n^2)$$

---

21. Pseudocode

```
shortestPathBinaryMatrix(grid):

    n = grid size

    if start blocked OR destination blocked:
        return -1

    directions =
        all 8 neighboring directions

    queue =
        [(start_row, start_col, path_length=1)]

    mark start visited
```

```
    while queue is not empty:

        remove front cell

        if current cell is destination:
            return path_length

        for each of the 8 directions:

            calculate neighbor

            if neighbor:
                is inside grid
                AND is open
                AND has not been visited

                    mark visited immediately
                    add neighbor with path_length + 1

    return -1
```

---

22. Python solution

```
from collections import deque
from typing import List


class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        n = len(grid)

        if grid[0][0] == 1 or grid[n - 1][n - 1] == 1:
            return -1

        directions = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),           (0, 1),
            (1, -1),  (1, 0),  (1, 1),
        ]

        queue = deque([(0, 0, 1)])

        # Reuse the grid itself as the visited structure.
        grid[0][0] = 1

        while queue:
            row, col, distance = queue.popleft()

            if row == n - 1 and col == n - 1:
                return distance
```

```
            for dr, dc in directions:
                nr = row + dr
                nc = col + dc

                if (
                    0 <= nr < n
                    and 0 <= nc < n
                    and grid[nr][nc] == 0
                ):
                    # Mark when enqueued, not when dequeued.
                    grid[nr][nc] = 1
                    queue.append((nr, nc, distance + 1))

        return -1
```

---

23. Non-obvious logic

Why start `distance = 1`?

The path length counts cells.

For:

```
[0]
```

the answer is `1`.

If the question instead asked for the number of **edges/moves**, we could start at `0`.

Always verify what "distance" means in the problem.

Why mutate `grid`?

Instead of creating:

```
visited = set()
```

we convert visited open cells:

```
0 -> 1
```

This saves an additional visited data structure.

The trade-off is important:

> The input matrix is modified.

If callers expect `grid` to remain unchanged, use a separate `visited` matrix/set instead.

Why mark visited before enqueueing?

Consider:

```
      A
     / \
    B   C
     \ /
      D
```

Both `B` and `C` could discover `D`.

If we wait to mark `D` until dequeuing:

```
queue could contain:
[D, D]
```

Instead:

```
grid[nr][nc] = 1
queue.append(...)
```

makes the first discovery claim the cell.

Because BFS's first discovery already represents a shortest path, later discoveries are unnecessary.

---

24. Alternative level-based implementation

You don't have to store the distance on every queue entry.

BFS naturally processes levels.

```
from collections import deque


class Solution:
    def shortestPathBinaryMatrix(self, grid):
        n = len(grid)

        if grid[0][0] != 0 or grid[n - 1][n - 1] != 0:
            return -1

        directions = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),           (0, 1),
            (1, -1),  (1, 0),  (1, 1),
        ]

        queue = deque([(0, 0)])
        grid[0][0] = 1
```

```
        distance = 1

        while queue:
            level_size = len(queue)

            for _ in range(level_size):
                row, col = queue.popleft()

                if row == n - 1 and col == n - 1:
                    return distance

                for dr, dc in directions:
                    nr = row + dr
                    nc = col + dc

                    if (
                        0 <= nr < n
                        and 0 <= nc < n
                        and grid[nr][nc] == 0
                    ):
                        grid[nr][nc] = 1
                        queue.append((nr, nc))

            distance += 1

        return -1
```

Both approaches are valid.

I generally prefer storing distance with each state when the state itself may later need extra metadata. The level-size version makes the **BFS-level invariant** particularly clear.

---

25. Queue invariant for this problem

At the beginning of each BFS level:

```
queue contains all currently discovered
cells at the same shortest-path distance
```

After processing them:

```
queue contains cells exactly one step farther
```

So:

```
distance 1:
start

distance 2:
```

```
all valid immediate neighbors

distance 3:
all newly reachable cells

...
```

This is why the first destination encounter is optimal.

---

26. Common BFS mistakes

Mistake 1 — Using DFS for an unweighted shortest path

DFS can find **a** path.

It does not naturally find the shortest path.

Mistake 2 — Forgetting `visited`

Can cause:

```
cycles
duplicate processing
huge runtime
```

Mistake 3 — Marking visited too late

Bad:

```
node = queue.popleft()
visited.add(node)
```

Better:

```
visited.add(neighbor)
queue.append(neighbor)
```

Mistake 4 — Using BFS for weighted edges

If:

```
edge A = cost 1
edge B = cost 50
```

normal BFS is no longer enough.

Think Dijkstra.

Mistake 5 — Using `list.pop(0)`

```
queue.pop(0)
```

is (O(n)).

Prefer:

```
deque.popleft()
```

which is (O(1)).

Mistake 6 — Wrong grid direction set

Four-direction BFS:

```
  ↑
← X →
  ↓
```

Eight-direction BFS:

```
↖ ↑ ↗
← X →
↙ ↓ ↘
```

Read the movement rules carefully.

---

27. BFS patterns worth recognizing

Once basic BFS is comfortable, most problems become variations of a few patterns.

Pattern 1 — Single-source BFS

```
one start
   ↓
all shortest distances
```

Example:

```
shortest path through grid
```

Pattern 2 — Multi-source BFS

Start with several nodes in the queue simultaneously.

```
S1     S2     S3
 \      |      /
```

```
BFS wavefront
```

Useful for:

- Rotting Oranges,
- nearest zero,
- nearest hospital,
- spreading fire/infection.

Pattern 3 — Level-order BFS

```
level 0
level 1
level 2
```

Useful for trees and minimum-step problems.

Pattern 4 — BFS with state

Queue entries may contain more than a node:

```
(node, distance)
```

or:

```
(row, col, keys_mask)
```

or:

```
(node, remaining_eliminations)
```

The important consequence is:

> `visited` may also need to represent the complete state, not only the node.

---

28. BFS versus related algorithms

Problem	Preferred approach
Tree level order	BFS
Unweighted shortest path	BFS
Equal edge weights	BFS
Multi-source nearest distance	BFS
Edge weights 0 or 1	0-1 BFS
Non-negative arbitrary weights	Dijkstra
Negative edges	Bellman-Ford / specialized approach

Problem	Preferred approach
Need any reachable path	DFS or BFS
Topological dependencies	Kahn's BFS or DFS

---

29. Senior interview explanation

A concise explanation would be:

> “I model each valid grid cell as a vertex and each legal movement as an unweighted edge. Because every movement has equal cost, BFS is appropriate: its FIFO queue processes states in nondecreasing shortest-path distance. I mark a cell visited when it is enqueued, which ensures each state is queued at most once and avoids duplicate work. Therefore the first time the destination is dequeued or discovered at its BFS level, the associated distance is optimal. With (n^2) cells and at most eight outgoing edges per cell, time and space are both (O(n^2)).”

That explanation demonstrates much more understanding than simply saying:

> “BFS uses a queue.”

Day 12 BFS takeaway

Remember this decision rule:

```
Need minimum steps / hops?
          |
          v
Are all moves the same cost?
       /       \
     Yes         No
      |           |
     BFS     weighted shortest
              path algorithm
```

And remember the central invariant:

$$\boxed{\text{BFS processes nodes in nondecreasing distance from the source}}$$

That one property explains **tree level order, unweighted shortest paths, multi-source spreading, and minimum-step grid problems**.